

Roteiro do vídeo demonstrativo (máx. 15 min)

Iago Roberto Esmério Almeida — PPGCC/UFPI

Fala

Gancho (transição)

comando

Requisitos do enunciado: (1) apresentação inicial em 2 min; (2) demonstração ao vivo no terminal em 8 min, com o Controlador-v2 e a árvore no Mininet, o `ping -f` levando o `top` a 100% de CPU e o `cbench` mostrando responses/sec; (3) análise dos resultados e defesa das 3 refatorações em 5 min.

Bloco	Tempo	Acumulado
1. Apresentação	2:00	2:00
2.1 Controlador + árvore no Mininet	1:30	3:30
2.2 <code>pingall</code> + <code>ping -f</code> + <code>top</code> a 100%	3:00	6:30
2.3 <code>cbench</code>	3:00	9:30
3. Gráficos e refatorações	5:00	14:30
Encerramento	0:30	15:00

Grave com folga: o ideal é fechar em ~14 min. O bloco 3 ficou mais longo porque agora explica cada gráfico; para compensar, o passo 4 do `cbench` (refatoração ao vivo) é o primeiro a ser cortado se o relógio apertar.

Neste arquivo, as **falas** aparecem como citações (>), os **comandos** em blocos de código e as indicações de **onde apontar** na tela em *itálico*. As frases em negrito no fim de cada bloco são os **ganchos**: servem para amarrar uma parte à seguinte, sem silêncio enquanto você troca de janela.

0. Preparação (antes de apertar o REC)

Ensaio obrigatório

O README avisa que a carga com `ping -f -l` (preload) **não foi testada**. Faça um ensaio completo do bloco 2.2 antes de gravar e confira se o `top` passa de 97%. Se não passar, use o plano B (seção 2.2, passo 5).

Limpeza do ambiente

```
cd ~/TA-Network/Trabalho      # pasta do trabalho (todos os comandos partem daqui)
sudo -v                       # deixa o sudo em cache, para não digitar senha no
↪ vídeo
sudo mn -c                     # remove restos de execuções anteriores do Mininet
ss -ltnp | grep 6653           # NÃO deve mostrar nada (porta livre)
killall -f Controlador         # se mostrou algo, mate o processo e confira de novo
```

Layout dos terminais

Deixe três terminais lado a lado, com fonte grande. Em **cada um** deles, entre na pasta do trabalho antes de qualquer outro comando:

```
cd ~/TA-Network/Trabalho
pwd                      # deve mostrar /home/iagora/TA-Network/Trabalho
ls                        # deve listar Controlador-v2.py, topologia_arvore.py,
↪ refatoracoes/ ...
```

Terminal	Função
T1 (esquerda)	Controlador
T2 (centro)	Mininet, depois cbench
T3 (direita, em cima)	top / htop

Deixe também abertos, para o bloco 3:

- o artigo/artigo.pdf (ou as figuras em artigo/figuras/*.png);
- o Controlador-v2.py no editor, já posicionado nas funções `recv_exact()`, `handshake()` e `main()` (laço principal com `listen(5)`).

```
cd ~/TA-Network/Trabalho/artigo
explorer.exe artigo.pdf          # no WSL abre no leitor do Windows (no Linux nativo:
↪ xdg-open artigo.pdf)
explorer.exe figuras             # pasta com as figuras exp1_*, exp2_*, exp3_*, exp4_*
cd ~/TA-Network/Trabalho
code Controlador-v2.py refatoracoes/ # editor com o controlador e as refatorações
```

Dica para o bloco 3: abrir os PNGs da pasta `figuras` em tela cheia, um por vez, facilita apontar com o mouse as curvas e os eixos.

1. Apresentação inicial (2 min)

Tela: primeira página do artigo (título, autor, resumo).

Olá! Eu sou Iago Roberto Esmério Almeida, aluno do Programa de Pós-Graduação em Ciência da Computação da UFPI. Este vídeo faz parte da primeira avaliação da disciplina de Tópicos Especiais em Redes de Computadores, com o professor Rayner Gomes Sousa.

O ponto de partida é uma característica central de SDN: como o plano de controle fica separado do plano de dados, o controlador entra no caminho crítico de cada novo fluxo. Se ele for lento, a rede inteira fica lenta. Então a pergunta que guia o trabalho é simples: até onde um controlador aguenta, e qual trecho do código é responsável por cada limite?

O controlador analisado é o `Controlador-v2.py`, um controlador minimalista que implementa o OpenFlow 1.0 direto sobre sockets TCP, em Python puro e com uma única thread. Três decisões de projeto dele vão aparecer o tempo todo neste vídeo: ele varre os sockets por *polling*; faz o handshake de forma síncrona, um switch por vez; e se comporta como um *hub*, respondendo a todo `PACKET_IN` com um `PACKET_OUT` de inundação, sem instalar regras. Ou seja, todo pacote da rede passa por ele.

Para medir esses limites, são avaliados quatro experimentos no Mininet com Open vSwitch, numa árvore de profundidade 2 e fanout 4, com 5 switches e 16 hosts, e usei também o cbench. O primeiro estressa o handshake com conexões simultâneas; o segundo inunda o controlador de `PACKET_IN` com `ping -f`; o terceiro mede a vazão bruta no cbench; e o quarto compara o original com três refatorações que implementei, usando `selectors`, `threads` e `asyncio`.

Gancho → E a melhor forma de entender esses limites é vê-los acontecer. Então vamos para o terminal.

2. Demonstração ao vivo no terminal (8 min)

2.1 Inicialização do controlador e da árvore (≈ 1:30)

Passo 1, T1: subir o controlador no modo padrão (verboso).

```
cd ~/TA-Network/Trabalho
python3 Controlador-v2.py
```

Saída esperada: === Controlador SDN Didático v2 ... e Aguardando conexões dos switches (porta 6653)...

No primeiro terminal, inicio o controlador. Ele abre um socket TCP na porta 6653, a porta padrão do OpenFlow, com `listen(5)`. Guardem esse número: ele significa que a fila de conexões pendentes comporta no máximo cinco, e isso vai fazer diferença daqui a pouco. Neste modo, ele também imprime uma linha para cada `PACKET_IN`, o que tem seu custo.

Com o controlador esperando, falta a rede.

Passo 2, T2: subir a árvore no Mininet.

```
cd ~/TA-Network/Trabalho
sudo mn --topo tree,depth=2,fanout=4 \
    --controller remote,ip=127.0.0.1,port=6653 \
    --switch ovsk,protocols=OpenFlow10
```

(Equivalente, de dentro de `~/TA-Network/Trabalho`: `sudo python3 topologia_arvore.py`.)

No segundo terminal, subo a árvore no Mininet: cinco switches Open vSwitch apontando para o controlador. Agora reparem no que aparece no primeiro terminal.

Passo 3, T1: apontar o log do handshake. Role o terminal até o início, se for preciso.

Este é o Experimento 1 acontecendo ao vivo. O log é estritamente sequencial: para cada switch aparecem a conexão TCP, o HELLO, o `FEATURES_REQUEST` e o `FEATURES_REPLY` com o DPID e o número de portas. Só quando um switch termina é que o próximo é aceito. Cada handshake leva de 2 a 4 milissegundos, e os cinco concluíram.

O “Handshake falhou” no início não é um switch, é a sonda com que o Mininet testa a porta. E o “PORT_STATUS ignorada” é uma mensagem que chegou enquanto o controlador estava preso negociando com outro switch.

Aqui funcionou porque o Mininet cria os switches um de cada vez. Quando as conexões chegam juntas, o `handshake()` bloqueia o laço inteiro, e com 16 switches a latência passa de 4 segundos. Vou mostrar isso no gráfico, na análise.

Logo depois aparecem linhas de `PACKET_IN` (tráfego IPv6 de descoberta de vizinhos dos hosts).

E, sem que eu tenha mandado nada, já começam os PACKET_IN: como o hub não instala regras, até o tráfego de descoberta de vizinhos dos hosts sobe para o controlador.

Gancho → Se até esse tráfego de fundo passa pelo controlador, a pergunta natural é: o que acontece quando eu realmente carrego a rede? É o Experimento 2.

2.2 pingall, ping -f e o top a 100% (≈ 3:00)

Passo 1, T3: abrir o top só com o processo do controlador.

```
cd ~/TA-Network/Trabalho
top -d 1 -p $(pgrep -f Controlador-v2.py)
```

(Se preferir o htop, que é visualmente mais claro: `htop -p $(pgrep -f Controlador-v2.py)`.)

Para acompanhar a carga, deixo o `top` no terceiro terminal, monitorando só o processo do controlador. Notem que, mesmo em repouso, ele já gasta alguns por cento de CPU: o laço acorda a cada milissegundo para varrer os sockets, tendo ou não algo para fazer.

Antes de estressar, preciso garantir que a rede funciona.

Passo 2, T2 (Mininet): conectividade.

```
mininet> pingall
```

Esperado: Results: 0% dropped (240/240 received).

O `pingall` testa todos os pares de hosts e dá zero por cento de perda. A rede está correta, com cada pacote passando pelo controlador. Agora sim, vamos estressá-lo.

Passo 3, T2: o comando exato do enunciado.

```
mininet> h1 ping -f h16
```

Deixe rodar uns 15 a 20 s, mostrando o `top`, e depois pare com **Ctrl+C**.

Este é o comando do enunciado: `h1 ping -f h16`, um ping em modo de inundação entre as duas pontas da árvore. Olhem o `top`: a CPU sobe, mas para em torno de 20 a 30%, e não em 100%. No experimento, a média foi de 23%.

Por quê? Porque o `ping -f` é autorregulado: só envia um novo pacote quando chega a resposta do anterior, ou a cada 10 milissegundos. Passando pelo controlador em três switches, o RTT fica em torno de 9 milissegundos, o que limita a carga a pouco mais de 100 pacotes por segundo. Ou seja, a própria lentidão do controlador segura a carga.

Ao parar, mostre as estatísticas (`packets transmitted, % packet loss, rtt`).

Então, para levá-lo de fato à exaustão, preciso manter vários pacotes em voo ao mesmo tempo.

Passo 4, T2: ping -f com preload, para chegar a 100%.

```
mininet> h1 ping -f -l 64 h16
```

Deixe rodar 30 a 40 s com a câmera no `top` e no T1 (o log rolando sem parar). Pare com **Ctrl+C**.

Com a opção `-i 64`, o ping mantém 64 pacotes em voo em vez de um. E agora o `top` mostra o controlador em 100% de um núcleo: ele está saturado. No experimento de saturação, isso acontece a partir de cerca de 3 mil pings por segundo. Como cada ping gera uns 10 `PACKET_IN`, por causa da inundação nos switches do caminho, isso equivale a cerca de 30 mil `PACKET_IN` por segundo. Esse é o teto do controlador.

E o tráfego? O excedente se acumula nas filas, o RTT sobe de 9 milissegundos para mais de um segundo e a perda cresce. Mas os switches não desconectam: o controlador segue respondendo aos `ECHO_REQUEST` no mesmo laço. A degradação aparece como perda e latência, não como queda do canal OpenFlow.

Ao parar, mostre a perda e o RTT no resumo do ping.

Passo 5 (plano B, só se o passo 4 não passar de ~95% no ensaio). Injeção em taxa fixa, igual ao `exp2_packet_in.py` taxa: 16 pings simultâneos de 250 req/s cada (≈ 4 mil ICMP/s), que param sozinhos em 40 s.

```
mininet> py [net.get('h%d' % i).popen('ping -q -i 0.004 -w 40 10.0.0.%d' % (17 - i))
↪ for i in range(1, 17)]
```

Como o ping `-f` se autorregula, faço o mesmo que no experimento de saturação: dezesseis pings em paralelo, em taxa fixa, somando cerca de 4 mil pacotes por segundo. E o `top` vai a 100%.

Se precisar interromper antes dos 40 s: `mininet> sh pkill -f "ping -q -i"`.

Passo 6: confirmar que os switches seguem conectados (opcional, 10 s).

```
mininet> sh ovs-vsctl show | grep is_connected
```

Esperado: 5 linhas `is_connected: true`.

E aqui está a confirmação: os cinco switches continuam conectados.

Passo 7: encerrar o Mininet e o controlador.

```
mininet> exit
```

No T1: **Ctrl+C**.

Gancho → O Mininet mostrou o limite com tráfego realista, mas ali o gargalo se mistura com o custo dos switches e dos hosts. Para medir só o controlador, isolado, a ferramenta padrão é o `cbench`. É o Experimento 3.

2.3 cbench: vazão em responses/sec ($\approx 3:00$)

Passo 1, T1: subir o controlador em modo silencioso.

```
cd ~/TA-Network/Trabalho
python3 Controlador-v2.py --quiet
```

Reinício o controlador com `--quiet`, que desliga o print por mensagem. Assim meço a arquitetura do controlador, e não a velocidade do terminal.

Passo 2, T2: o comando exato do enunciado, com 8 switches.

```
cd ~/TA-Network/Trabalho
cbench -c 127.0.0.1 -p 6653 -m 10000 -l 10 -s 8
```

(Se o `cbench` não estiver no `PATH`, use o binário da pasta `ferramentas/`, a partir de `~/TA-Network/Trabalho`: `./ferramentas/cbench -c 127.0.0.1 -p 6653 -m 10000 -l 10 -s 8`.)

Esperado: falha imediata com `make_tcp_connection: connect: Operation now in progress` e `make_nonblock_tcp_connection: returned -1`.

Este é o comando do enunciado: 8 switches emulados mandando `PACKET_IN`, em 10 laços de 10 segundos. E ele falha na hora.

Lembram do `listen(5)` e do `handshake` um switch por vez? É isso. O `cbench` abre as 8 conexões ao mesmo tempo, com `connect()` não bloqueante, e trata como erro qualquer conexão que não se complete na hora. Com o controlador preso num `handshake` e só cinco lugares na fila, a oitava fica pendente e o teste aborta, em todas as execuções. É o gargalo do Experimento 1, agora impedindo o próprio benchmark de rodar.

Se o controlador ficou travado/girando (veja o `top`), reinicie-o no T1 (`Ctrl+C` e `python3 Controlador-v2.py --quiet`).

Então a pergunta passa a ser: qual o maior número de switches que ele aceita?

Passo 3, T2: com 7 switches, o maior N que funciona.

```
cbench -c 127.0.0.1 -p 6653 -m 10000 -l 10 -s 7
```

Leva ~100 s. Use esse tempo para falar, mostrando o `top` (no T3, refaça `top -d 1 -p $(pgrep -f Controlador-v2.py)`, porque o PID mudou).

Com 7 switches o teste completa, então essa é a vazão máxima que consigo relatar. Cada linha que aparece é um laço de 10 segundos, com o número de respostas por segundo. E no `top` a CPU fica perto de 90%.

Enquanto o teste roda, vale entender o que está sendo medido. No modo padrão do `cbench`, o de latência, cada switch emulado mantém uma única requisição pendente: manda um `PACKET_IN` e espera a resposta para mandar o próximo. Para o controlador, cada mensagem custa duas chamadas de `recv` e um `sendall`, mais o giro do *polling* sobre os sockets vazios, que lança uma exceção `BlockingIOError` a cada volta. Esse custo por mensagem é o que vai definir o resultado.

Ao terminar, aponte a linha final:

```
RESULT: 7 switches 9 tests min/max/avg/stddev = ... responses/s
```

E o resultado: cerca de 108 mil respostas por segundo em média. São 9 testes porque o primeiro laço é aquecimento e é descartado. Para ter uma referência, na literatura o Beacon passa de um milhão de respostas por segundo com uma única thread, e o ONOS chega a centenas de milhares.

Passo 4 (opcional, se o relógio permitir ≈ 45 s): a refatoração com `selectors` aguenta 8 switches.

T1: `Ctrl+C` e:

```
cd ~/TA-Network/Trabalho
python3 refatoracoes/Controlador-v3-selectors.py
```

T2:

```
cbench -c 127.0.0.1 -p 6653 -m 10000 -l 3 -s 8
```

E se o problema é a arquitetura, mudar a arquitetura deveria resolver. Aqui está a refatoração baseada em `selectors` rodando o mesmo teste com 8 switches: ele roda normalmente, a cerca de 125 mil respostas por segundo. Reduzi para 3 laços só para caber no tempo do vídeo.

Ao final, Ctrl+C no T1 e confira que a porta ficou livre (`ss -ltnp | grep 6653`).

Gancho → Até aqui vimos os sintomas ao vivo: o handshake sequencial, a CPU a 100% e o cbench falhando com 8 switches. Agora vamos aos números das rodadas completas, para ver o comportamento de cada limite e como as três refatorações se saem.

3. Análise dos resultados e defesa arquitetural (≈ 5:00)

Tela: figuras em tela cheia (ou o artigo em PDF), uma por vez. Para cada gráfico: diga em uma frase o que está em cada eixo, depois aponte com o mouse a curva que está descrevendo. O ritmo aqui é apertado (≈ 750 palavras para 5 min): se atrasar, corte primeiro o parágrafo do microbenchmark e o gráfico de CPU do Experimento 4 (`exp4_cpu`).

```
cd ~/TA-Network/Trabalho/artigo
explorer.exe artigo.pdf          # ou: xdg-open artigo.pdf
```

Figura do Experimento 1 (`exp1_handshake`), ≈ 1:00

Onde apontar: eixo X e eixo Y (escala log); a curva vermelha nos dois painéis, com o salto de 1 para 2 switches no painel (b); depois as curvas azul e rosa, quase planas, embaixo.

Começando pelo handshake. Nos dois gráficos, o eixo horizontal é o número de switches conectando juntos, e o vertical é o tempo em escala logarítmica, ou seja, cada linha de grade vale dez vezes a anterior. Em vermelho, o original.

À esquerda, a latência total: a curva vermelha está sempre no topo e sobe sem parar, de 0,3 segundo com um switch para 4,3 segundos com dezesseis. Cada switch novo espera todos os anteriores.

À direita, só a espera até o controlador mandar o primeiro HELLO. Com um switch, menos de um milissegundo; com dois, a curva salta para 200; com dezesseis, um segundo. Esse salto logo no segundo switch é a assinatura do handshake síncrono: enquanto atende um, o laço não atende ninguém. Com dezesseis, a fila do `listen(5)` estoura e, em média, 9 conexões por rodada são abortadas.

Nas refatorações, as curvas ficam praticamente planas e o HELLO sai em cerca de um milissegundo. O degrau verde e o pico rosa à esquerda vêm do próprio Open vSwitch; o painel do HELLO mostra que o controlador não tem culpa.

Gancho → Esse é o limite de conexões. E com todos conectados, quanto tráfego ele aguenta?

Figuras do Experimento 2 (`exp2_ping_flood` e `exp2_saturacao`), $\approx 1:15$

Onde apontar (`exp2_ping_flood`): as faixas no topo (ocioso, pingall, ping -f, pós-flood) e o patamar da linha vermelha.

Esta é a série temporal do que mostrei ao vivo: tempo no eixo horizontal, CPU no vertical, controlador em vermelho. Ocioso, ele já gasta de 5 a 10%, o custo do *polling*. Com o ping -f, sobe para um patamar plano em torno de 23% e volta ao fim. Esse patamar plano é a autorregulação do ping.

Onde apontar (`exp2_saturacao`): eixo X; curvas vermelha e azul subindo juntas; a faixa rosa; a azul achatando e a preta decolando logo depois.

Por isso, aqui eu controlei a taxa, no eixo horizontal. A CPU está em vermelho, a perda em preto, e os PACKET_IN processados em azul, no eixo da direita.

Até 2,6 mil pings por segundo, vermelho e azul sobem juntos, quase em linha reta, e a perda é zero: tudo que chega é atendido. Na faixa rosa acontece a virada: a CPU encosta em 97%, a curva azul achata em torno de 30 mil PACKET_IN por segundo, e a perda começa a subir. Dali em diante, mais carga não gera mais trabalho, só mais perda. Esse joelho é a saturação.

E o que define esse teto no código? Pelo microbenchmark, não é o `struct.unpack()`, só 1% do custo. É a E/S: duas leituras e uma escrita custam 3,4 microssegundos por mensagem, contra 63 nanossegundos quando um único `recv` traz 64 mensagens.

Gancho → Se o custo está na E/S por mensagem, o cbench deve mostrar o mesmo padrão.

Figura do Experimento 3 (`exp3_cbench`), ≈ 40 s

Onde apontar: no painel (a), as linhas planas e empilhadas; no (b), as barras subindo e desacelerando depois de 5, a linha da CPU e as marcas “falha”.

À esquerda, cada linha é uma rodada com um número de switches, ao longo do tempo do teste. Elas são planas, ou seja, a vazão é estável, e se empilham: mais switches, linha mais alta.

À direita, as barras são a vazão média e a linha tracejada é a CPU. A vazão vai de 30 mil com um switch a 108 mil com sete: com mais switches, cada volta do laço encontra mais dados e o custo do *polling* se dilui. Mas, depois de cinco, o crescimento desacelera, porque a CPU já está perto de 90% e não há mais folga. Em 8 e 16, nenhuma barra: é

a falha que vimos ao vivo. Como a aplicação é um simples hub, essa distância para os controladores de produção vem da arquitetura, e não da lógica de controle.

Gancho → E é essa arquitetura que as três refatorações atacam.

As três refatorações + Experimento 4 (`exp4_*`), $\approx$ 2:00

Se quiser, mostre o código lado a lado:

```
cd ~/TA-Network/Trabalho/refatoracoes
ls                               # of10.py, Controlador-v3-selectors.py, -threads.py,
↪ -asyncio.py
code of10.py Controlador-v3-*.py
```

As três mantêm o hub e compartilham duas mudanças contra o custo de E/S: cada conexão lê para um buffer e extrai todas as mensagens completas de uma vez, e as respostas de um lote saem num único `send`. A **primeira** troca o *polling* pelo `selectors`, que dorme no kernel até haver dados, com o handshake como máquina de estados e backlog de 128. A **segunda** usa uma thread por switch. A **terceira** usa `asyncio`, com uma fila limitada entre leitura e despacho, que dá *backpressure*, no modelo do Ryu.

Onde apontar (*exp4_vazao_cbench*): azul acima da vermelha; vermelha terminando em 7, com os X em 8 e 16; verde caindo depois de 2; rosa plana embaixo.

Aqui, switches no eixo horizontal e vazão no vertical, no modo do enunciado. A curva azul, do `selectors`, fica acima da vermelha, 14% a mais com sete switches, e continua onde a vermelha para: cerca de 125 mil com 8 e 16 switches, que o original nem aceita.

As outras duas surpreendem. A verde, de `threads`, sobe até dois switches e depois cai para uns 10 mil: com uma única mensagem pendente por switch, cada resposta exige passar o GIL de uma thread para outra. A rosa, do `asyncio`, fica plana entre 12 e 21 mil, presa ao custo fixo do *event loop* por mensagem.

Opcional (corte se estiver atrasado). Onde apontar (*exp4_cpu*): a barra hachurada “2” do base; a barra verde de 153.

Na CPU, as barras claras são o repouso: só o original gasta CPU parado. Sob carga, todos ficam perto de 100%, menos as threads, que passam de 150% para entregar dez vezes menos.

Onde apontar (*exp4_throughput*): a barra vermelha quase invisível em 7 e “falha” em 8; as demais encostando em 1 milhão.

Por último, o modo *throughput*, em que os switches não esperam resposta. O quadro muda: as três passam de 970 mil respostas por segundo, mais de 20 vezes o original, cuja barra vermelha mal aparece. Com mensagens acumuladas, cada `recv` traz dezenas delas e o lote amortiza a E/S.

Encerramento (≈ 30 s)

Para concluir: os limites do Controlador-v2 vêm da arquitetura de E/S, e não da decodificação com `struct`. O handshake síncrono com `listen(5)` impede a escala de conexões, e o *polling* com várias syscalls por mensagem limita a vazão. São exatamente os problemas que levaram frameworks como o Ryu a adotar modelos assíncronos, orientados a eventos, com E/S em lote e filas.

E a lição extra das medições é que trocar o modelo de concorrência, sozinho, não basta: as threads e o `asyncio` resolveram as conexões, mas só ganharam vazão quando havia mensagens acumuladas. É preciso olhar o custo por mensagem e o regime de carga. Obrigado!

Problemas comuns durante a gravação

Sintoma	Causa provável	O que fazer
Controlador: Address already in use	Processo antigo na 6653	<code>pkill -f Controlador; ss -ltnp grep 6653</code>
Mininet não sobe / interfaces duplicadas	Restos da execução anterior	<code>sudo mn -c</code>
<code>top -p</code> vazio ou com erro	PID mudou ao reiniciar o controlador	Rode o <code>top -p \$(pgrep -f ...)</code> de novo
<code>top</code> não chega a 100% com <code>-l 64</code>	O terminal não acompanha o volume de prints, ou a carga é baixa	Aumente para <code>-l 128</code> ou use o plano B (passo 5)
<code>cbench -s 7</code> também falha	Controlador ficou girando após o <code>-s 8</code>	Reinicie o controlador antes de cada execução do <code>cbench</code>
<code>cbench: command not found</code>	Fora do PATH	<code>cd ~/TA-Network/Trabalho e ./ferramentas/cbench ...</code>
No such file or directory ao rodar <code>python3 ...</code>	Terminal fora da pasta do trabalho	<code>cd ~/TA-Network/Trabalho e confira com <code>pwd</code></code>

Depois de gravar

1. Suba o vídeo (YouTube não listado ou Google Drive com acesso de visualização) e confira o link numa janela anônima.
2. Coloque o link no `artigo.tex`, no lugar dos dois `\todo{link público} / \todo{public link}` do resumo e do abstract, e recompile: `cd ~/TA-Network/Trabalho/artigo && latexmk -pdf artigo.tex`.
3. Confira a duração final: no máximo 15:00.